

chapter-1-basic-prompt-structure

May 15, 2025

1 Chapter 1. Basic Prompt Structure

Upstage offers various types of APIs, including Chat, Text Embedding, Translation, Grounding Check, Layout Analysis, Key Information Extraction, and Document Processing. In this book, we will exclusively focus on using the `Chat` API.

For more information about the APIs, please refer to the following [link](#).

1.1 Table of Contents

- Use `Ctrl + F` (Windows) or `Cmd + F` (Mac) to locate specific sections by title.
 - **1.1 Chat API**
 - **1.2 Understanding Parameters**
 - 1.2.1 Paramters
 - 1.2.2 Examples
 - **1.3 Understanding Structure**
 - 1.3.1 Input Structure
 - 1.3.2 Example
 - **1.4 Understanding System Prompt**
 - 1.4.1 System Prompt
 - 1.4.2 Examples
 - 1.4.3 Practice
-

1.2 1.1 Chat API

The Following is a standard API call format used to interact with Upstage's API for generating chat completions.

```
[ ]: from openai import OpenAI

# Retrieve the UPSTAGE_API_KEY variable from the IPython store
%store -r UPSTAGE_API_KEY
```

```

try:
    if UPSTAGE_API_KEY:
        print("Success!")
except NameError as ne:
    print(f"Since, {ne}")
    print("Please, insert your API key.")
    UPSTAGE_API_KEY = input("UPSTAGE_API_KEY = ")

# Set your API key:
# UPSTAGE_API_KEY = " " ← Insert your API key here.

client = OpenAI(
    api_key= UPSTAGE_API_KEY,
    base_url="https://api.upstage.ai/v1/solar"
)

response = client.chat.completions.create(
    model="solar-pro",
    messages=[
        {
            "role": "user",
            "content": "Describe how we plan to leverage Upstage products to
↪achieve your mission of AGI for Work."
        }
    ],
)

print('\n', response.choices[0].message.content, '\n\n')

```

Success!

Upstage's products, UPstage Sage and UPstage Jurassic, can play a significant role in achieving our mission of AGI for Work. Here's how:

1. UPstage Sage: This AI Document Processing tool can help us manage and analyze vast amounts of data. By extracting relevant information from documents, we can feed this data into our AGI systems for training and learning. This will enhance the AGI's ability to understand, interpret, and generate human-like text, thereby improving its performance in various work-related tasks.
2. UPstage Jurassic: Our customizable, no-code AI solutions can be used to build and deploy AGI systems tailored to specific industries or tasks. By leveraging Jurassic, we can create AGI systems that can understand and perform a wide range of tasks, from customer service to data analysis, thereby increasing efficiency and productivity in the workplace.

3. Continuous Learning and Improvement: Both Sage and Jurassic can be used in a continuous learning loop. As the AGI systems learn and improve, they can process more complex tasks, which in turn generates more data for Sage to analyze. This data can then be used to further train and improve the AGI systems, creating a cycle of continuous learning and improvement.

4. Scalability: With Upstage's products, we can scale our AGI systems to handle increasing workloads and complexities. This scalability is crucial for achieving our mission of AGI for Work, as it allows us to serve a wider range of industries and tasks.

In summary, Upstage's products can help us build, train, and scale AGI systems for work, thereby achieving our mission of AGI for Work.

1.3 1.2 Understanding Parameters

When performing prompt engineering, parameters are key to controlling how the model behaves and the type of output you receive. Here's a detailed explanation of these parameters and their role in the completion generation process.

1.3.1 1.2.1 Paramters

Model:

The specific model you are intending to interact with.

Max_Tokens:

- This parameter limits the total number of tokens (words or parts of words) in the output. Controlling `max_tokens` allows you to set a maximum length for the model's output. This is useful to avoid overly long responses, control API usage costs, or tailor the output for specific use cases (e.g., short answers, summaries, etc.).
- **Hard Stop:**
 - Prevents the model from generating tokens beyond the specific limit.
 - The generation may stop mid-word or mid-sentence when the token limit is reached.
- **Prompt Tokens:** The number of tokens in the input prompt.
- If `max_tokens` is set, the sum of input tokens and `max_tokens` must be less than or equal to the model's context length (4096)

Temperature:

This parameter controls the randomness or creativity of the model's responses.

- A higher value allows for more flexibility, resulting in more diverse text generation.
- A lower value makes the model more deterministic, typically generating more accurate and consistent output.

The valid range is between **0** and **2.0** (0 `Temperature` 2.0).

- **0.0:** The output is deterministic and predictable, meaning the model will likely return the same response to the same prompt every time.
- **0.7:** This is a balanced level, where the model is creative but still focused. The responses may vary, but they tend to stay on topic.
- **2.0:** This encourages highly creative or random output, potentially producing more unusual or diverse responses.

Top_P:

This is an alternative way to control the randomness of the model's output by considering the cumulative probability of token choices. **Top_P** allows you to control how “safe” or “risky” the model is in generating its response. Lower values reduce the model's sampling range, forcing it to stick to higher-probability tokens, while higher values increase diversity in the responses.

- **Top_P = 0.9:** the model will sample tokens from the smallest set whose cumulative probability is 90%.

! How it differs from temperature: While `temperature` affects how creative the model is overall, `Top_P` affects how many of the high-probability tokens are considered in the final response.

Summary

- **model:** Defines the specific AI model being used.
- **max_tokens:** Limits the length of the response.
- **temperature:** Controls the creativity or randomness of the response.
- **top_p:** Controls how many token choices the model considers based on probability.

1.3.2 1.2.2 Examples

Example #1: Configuration

```
[ ]: config_model = {  
    "model": "solar-pro",  
    "max_tokens": 2000,  
    "temperature": 0.7,  
    "top_p": 0.9,  
}
```

Example #2: Temperature and Top_P Adjustment

Objective: Compare how creativity and randomness affect responses.

```
[ ]: config_robust = {  
    "model": "solar-pro",  
    "messages": [  
        {  
            "role": "user",  
            "content": "What are the potential benefits of AI in healthcare?"  
        }  
    ]  
}
```

```

],
    "max_tokens": 2000,
    "temperature": 0.0,
    "top_p": 0.8
}

response = client.chat.completions.create(**config_robust)
print(response.choices[0].message.content, '\n\n')

```

AI has the potential to revolutionize healthcare in many ways. Here are a few potential benefits:

1. Improved Diagnosis: AI can analyze complex medical data and help doctors make more accurate diagnoses. For example, AI algorithms can analyze medical images to detect diseases like cancer at an early stage.
2. Personalized Treatment: AI can help doctors create personalized treatment plans based on a patient's genetic makeup, lifestyle, and other factors. This can lead to more effective treatments and better patient outcomes.
3. Drug Discovery: AI can help researchers discover new drugs and treatments faster and more efficiently. AI algorithms can analyze vast amounts of data to identify potential drug candidates and predict their effectiveness.
4. Remote Monitoring: AI-powered wearable devices and remote monitoring systems can help doctors keep track of patients' health in real-time. This can help doctors detect health issues early and provide timely interventions.
5. Administrative Efficiency: AI can help healthcare providers streamline administrative tasks, such as scheduling appointments, managing patient records, and processing insurance claims. This can help reduce costs and improve patient care.
6. Mental Health: AI-powered chatbots and virtual therapists can provide mental health support to patients who may not have access to traditional therapy. These tools can help patients manage stress, anxiety, and depression.
7. Predictive Analytics: AI can help healthcare providers predict health trends and identify potential health risks before they become serious issues. This can help providers take proactive measures to prevent diseases and improve patient outcomes.

```

[ ]: config_creative = {
    "model": "solar-pro",
    "messages": [

```

```

    {
        "role": "user",
        "content": "What are the potential benefits of AI in healthcare?"
    }
],
"max_tokens": 2000,
"temperature": 1.0,
"top_p": 1.0
}

response = client.chat.completions.create(**config_creative)
print(response.choices[0].message.content, '\n\n')

```

AI has the potential to transform various aspects of healthcare, bringing about numerous benefits. Here are some of them:

1. Improved Diagnosis: AI can help in early detection of diseases such as cancer by analyzing medical images with higher accuracy and consistency than human professionals.
2. Personalized Treatment: AI can analyze patient data to develop customized treatment plans, taking into account their medical history, genetics, lifestyle, and other factors.
3. More Accurate Predictions: AI can help predict outbreaks of diseases by analyzing patterns and trends in large sets of data, enabling better preparedness and prevention measures.
4. Better Drug Discovery: AI can speed up the process of drug discovery by analyzing vast amounts of data to identify potential compounds and predict their effects.
5. Improved Clinical Trials: AI can help identify suitable participants for clinical trials, analyze real-time results, and monitor symptoms, making the process more efficient and effective.
6. Enhanced Patient Engagement: AI-powered chatbots can help patients manage their health conditions by providing personalized advice, reminders, and answering their queries.
7. Reduced Healthcare Costs: By improving efficiency, reducing errors, and enabling early intervention, AI can help in cutting down healthcare costs.
8. Increased Access to Healthcare: AI can help bring healthcare services to remote or underserved regions by enabling remote diagnosis and monitoring.
9. Workflow Efficiency: AI can automate administrative tasks such as scheduling

appointments, managing patient records, and processing insurance claims, allowing healthcare professionals to focus on patient care.

10. Enhanced Research: AI can help analyze vast amounts of medical literature and data, assisting researchers in making new discoveries and advancing medical knowledge.

Remember, while AI holds great promise in healthcare, it also poses certain challenges and risks, such as data privacy and bias. It's important to address these issues as we embrace AI in healthcare.

Example #3: Limiting Output with Max_Tokens

Objective: Control the length of responses and stop them at specific points.

```
[ ]: config_output_400 = {  
    "model": "solar-pro",  
    "messages": [  
        {  
            "role": "user",  
            "content": "Explain how Upstage AI models handle natural language_  
↳processing. Explain it in a way that non-developers can easily understand."  
        }  
    ],  
    "max_tokens": 400,  
    "temperature": 0.7,  
}  
  
response = client.chat.completions.create(**config_output_400)  
print(response.choices[0].message.content, '\n\n')
```

Sure, I'd be happy to explain! Upstage is a company that provides AI solutions, and one of their key offerings is their natural language processing (NLP) models. These models are designed to understand, interpret, and generate human language in a way that's similar to how humans do it.

Here's a simple way to understand it: Have you ever used a voice assistant like Siri or Alexa? When you ask a question or give a command, these assistants use NLP to understand what you're saying. They break down your words into smaller parts, determine the meaning behind them, and then respond in a way that makes sense.

Upstage's NLP models work in a similar way. They take in a piece of text, like a sentence or a paragraph, and analyze it. They look at the words, the order of the words, the context, and other factors to understand what the text is saying. Then, they can generate a response or perform an action based on that understanding.

For example, if you asked Upstage's NLP model, "What's the weather like today?", it would analyze your question, determine that you're asking about the weather, and then provide you with the current weather conditions. It's like having a conversation with a really smart friend who knows a lot about different topics!

So, in a nutshell, Upstage's NLP models handle natural language processing by understanding the meaning behind human language and responding in a way that's both accurate and helpful.

```
[ ]: config_output_40 = {
    "model": "solar-pro",
    "messages": [
        {
            "role": "user",
            "content": "Explain how Upstage AI models handle natural language_
↳processing. Explain it in a way that non-developers can easily understand."
        }
    ],
    "max_tokens": 40,
    "temperature": 0.7,
}

response = client.chat.completions.create(**config_output_40)
print(response.choices[0].message.content, '\n\n')
```

Of course! Natural Language Processing (NLP) is a branch of AI that helps computers understand, interpret, and generate human language. Upstage's AI models handle NLP in

1.4 1.3 Understanding Structure

1.4.1 1.3.1 Input Structure

messages :

This is an array containing the conversation context. It includes exchanges between the user and the model. Each element contains:

- “role”:

The role can be "user", "assistant", or "system", indicating the source of the message.

In the case of "role": "system", it sets the behavior, tone, and knowledge base of the assistant, acting as an initial instruction.

In the case of "role": "user", it specifies that the message comes from the user.

In the case of "role": "assistant", it contains responses generated by the AI to address the user's queries or continue the conversation.

1.4.2 1.3.2 Example

```
[ ]: {  
    "role": "system",  
    "content": "You are my Assistant. Your role is to answer my questions_  
↳faithfully and in detail."  
}
```

```
[ ]: {  
    "role": "user",  
    "content": "Hello, Solar. Can you help me plan a weekend trip to New York_  
↳City?"  
}
```

```
[ ]: {  
    "role": "assistant",  
    "content": "Hello! I'd be happy to help you plan your weekend trip to New_  
↳York City. Let's start by discussing your interests and preferences. Are you_  
↳looking for sightseeing, shopping, diningor perhaps a mix of all?"  
}
```

1.5 1.4 Understanding System Prompt

1.5.1 1.4.1 System Prompt

The **system prompt** plays a key role in shaping how the AI model interprets and responds to user inputs. In the context of prompt engineering, understanding and utilizing the system prompt effectively can help guide the model's behavior and ensure that its responses are aligned with user expectations. In this book, we show **two different types of system prompts: a short version and a long version.**

Tips:

If the system prompt is short, the responses tend to be brief; if the system prompt is long, the responses are generally longer.

1.5.2 1.4.2 Examples

Example #1: Short Version

```
[ ]: config_model = {  
    "model": "solar-pro",  
    "max_tokens": 2000,
```

```

        "temperature": 0.7,
        "top_p": 0.9,
    }

    message = [
        {
            "role": "system",
            "content": "You are an AI assistant to help user's various tasks.␣
↳Please provide me with an accurate information."
        },
        {
            "role": "user",
            "content": "Explain about Blockchain in detail."
        }
    ]

    config = {**config_model, "messages": message}

    response = client.chat.completions.create(**config)
    print(response.choices[0].message.content, '\n\n')

```

Blockchain is a decentralized and distributed digital ledger that is used to record transactions across many computers in a secure and transparent manner. It is the underlying technology behind cryptocurrencies such as Bitcoin, but its potential uses extend far beyond that.

At its core, a blockchain is a chain of blocks, where each block contains a number of transactions. Once a block is added to the chain, it cannot be altered or deleted, making the blockchain a permanent and immutable record of all the transactions that have ever taken place on it.

One of the key features of a blockchain is its decentralization. Instead of being stored on a single central server, the blockchain is stored on a network of computers, or nodes, that are spread out across the world. This means that there is no single point of failure, and the blockchain can continue to function even if some of the nodes go offline.

Another important feature of a blockchain is its security. Because the blockchain is distributed across a network of nodes, it is very difficult for anyone to tamper with the data on it. In order to alter the blockchain, a hacker would need to gain control of a majority of the nodes in the network, which is practically impossible. Additionally, the use of cryptographic algorithms ensures that the data on the blockchain is encrypted and secure.

The use of blockchain technology has the potential to revolutionize a wide range of industries, including finance, healthcare, supply chain management, and more. For example, in the finance industry, blockchain can be used to facilitate

faster and more secure cross-border payments, reduce the risk of fraud, and lower the costs associated with intermediaries such as banks and other financial institutions. In the healthcare industry, blockchain can be used to securely store and share patient data, improve the accuracy of medical records, and streamline the process of clinical trials.

In summary, blockchain is a powerful and innovative technology that has the potential to transform the way we do business and interact with each other. Its decentralized, secure, and transparent nature makes it well-suited for a wide range of applications, and its potential uses are only limited by our imagination.

Example #2: Long Version

```
[ ]: config_model = {
    "model": "solar-pro",
    "max_tokens": 2000,
    "temperature": 0.7,
    "top_p": 0.9,
}

message = [
    {
        "role": "system",
        "content": "Your name is Solar. As my friendly AI language assistant,␣
↪you are tasked with providing me an accurate information. If you find that␣
↪the information at hand is inadequate, please ask me for further information.
↪ [Strong Rule] If you don't have any real-time information about the user's␣
↪query, please be honesty."
    },
    {
        "role": "user",
        "content": "Explain about Blockchain in detail."
    }
]

config = {**config_model, "messages": message}

response = client.chat.completions.create(**config)
print(response.choices[0].message.content, '\n\n')
```

Blockchain is a decentralized, distributed ledger technology that allows for secure, transparent, and tamper-proof recording of transactions. It was first introduced as the underlying technology for the cryptocurrency Bitcoin, but its potential uses extend far beyond that.

At its core, a blockchain is a chain of blocks, where each block contains a number of transactions. Once a block is added to the chain, it cannot be altered or deleted, making the entire history of transactions on the blockchain immutable.

Here are some key features and concepts related to blockchain:

1. Decentralization: A blockchain is maintained by a network of computers, rather than a central authority. This makes it more resistant to censorship, hacking, and other forms of interference.
2. Distributed Ledger: A blockchain is a shared database that is distributed across the network of computers. This means that every participant on the network has access to the same information, and can verify the accuracy of the data.
3. Consensus Mechanisms: In order to add a new block to the chain, the network must reach a consensus on its validity. This is typically done through a process called mining, where computers on the network compete to solve a complex mathematical problem. The first computer to solve the problem gets to add the new block to the chain, and is rewarded with a certain amount of cryptocurrency.
4. Smart Contracts: A smart contract is a self-executing contract with the terms of the agreement between buyer and seller being directly written into lines of code. They allow for the automation of complex processes, such as the transfer of assets, without the need for a third-party intermediary.
5. Cryptography: Blockchain uses advanced cryptography to secure the transactions on the network. Each block is linked to the previous block through a complex cryptographic hash function, making it nearly impossible to alter the data in any block without also altering all the subsequent blocks.

There are several types of blockchains, including public, private, and permissioned. Public blockchains, like Bitcoin and Ethereum, are open to anyone and allow for anonymous participation. Private blockchains, on the other hand, are restricted to a specific group of participants and allow for more control over the network. Permissioned blockchains are a hybrid of the two, allowing for some level of control over the network while also enabling participation from a wider group of users.

Blockchain has the potential to revolutionize a wide range of industries, from finance and banking to supply chain management and healthcare. Its ability to provide secure, transparent, and tamper-proof record-keeping can help to reduce fraud, increase efficiency, and build trust between parties. However, it also presents a number of challenges, such as scalability, energy consumption, and regulatory uncertainty.

In summary, blockchain is a powerful and innovative technology that has the potential to transform the way we conduct transactions and share information. Its decentralized, distributed, and cryptographic nature makes it secure, transparent, and tamper-proof, while its ability to enable smart contracts and support a wide range of use cases makes it a versatile tool for building the

next generation of digital applications.

1.5.3 1.4.3 Practice

Try switching between the short and long versions of the system prompt with different questions to experience the difference in responses.

Short System Prompt

```
[ ]: message = [  
    {  
        "role": "system",  
        "content": "You are an AI assistant to help user's various tasks.␣  
↪Please provide me with an accurate information."  
    },  
    {  
        "role": "user",  
        "content": " " # ← Insert your prompt here.  
    }  
]  
  
config = {**config_model, "messages": message}  
  
response = client.chat.completions.create(**config)  
print(response.choices[0].message.content, "\n\n")
```

Long System Prompt

```
[ ]: message = [  
    {  
        "role": "system",  
        "content": "Your name is Solar. As my friendly AI language assistant,␣  
↪you are tasked with providing me an accurate information. If you find that␣  
↪the information at hand is inadequate, please ask me for further information.  
↪ [Strong Rule] If you don't have any real-time information about the user's␣  
↪query, please be honesty."  
    },  
    {  
        "role": "user",  
        "content": " " # ← Insert your prompt here.  
    }  
]  
  
config = {**config_model, "messages": message}  
  
response = client.chat.completions.create(**config)  
print(response.choices[0].message.content, "\n\n")
```

Custom Your System Prompt

```
[ ]: config_your_model = {
    "model": "solar-pro",
    "max_tokens": 0.0, # ← Insert your max_tokens: 0.0 ~ here.
    "temperature": 0.0, # ← Insert your temperature: 0.0 ~ 2.0 here.
    "top_p": 0.0,      # ← Insert your top_p: 0.0 ~ 1.0 here.
}

message = [
    {
        "role": "system",
        "content": " " # ← Insert your system prompt here.
    },
    {
        "role": "user",
        "content": " " # ← Insert your prompt here.
    }
]

config = {**config_your_model, "messages": message}

response = client.chat.completions.create(**config)
print(response.choices[0].message.content, '\n\n')
```

Next: Chapter 2. Direct and Clear Prompts